UniCEUB - CENTRO UNIVERSITÁRIO DE BRASÍLIA

Faculdade de Tecnologia e Ciências Sociais Aplicadas

Curso: Ciência da Computação

Disciplina: Compiladores

Professor: THIAGO FERNANDES NEVES

Atividade avaliativa compiladores: (LAB-4)

Data entrega: 20/05/2025

Integrantes do grupo 7: Guilherme Nóbrega Gomes Dantas, Fernando Cajango Dias, Marco Vinícius Menezes Xavier, Gustavo Costa

Atividade 1

Criar as estruturas definidas no LAB4.

Compilar com sucesso utilizando o Makefile.

Fazer o teste com arquivo de entrada (test.c): 1234 + 6789.

lexer.c

```c
/* BEGIN - LAB 2 ----------------------------------*/
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <stdlib.h>
#include "helpers/vector.h"
#include "helpers/buffer.h"
#include "compiler.h"
```

```
#define LEX_GETC_IF(buffer, c, exp) \
    for (c = peekc(); exp; c = peekc()){ \
        buffer_write(buffer, c); \
        nextc(); \
    }

struct token* read_next_token();
static struct lex_process* lex_process;
static struct token tmp_token;

const char* keywords[] = {
    "auto", "break", "case", "char", "const", "continue", "default",
    "do", "double", "else", "enum", "extern", "float", "for", "goto",
    "if", "int", "long", "register", "return", "short", "signed",
    "sizeof", "static", "struct", "switch", "typedef", "union",
    "unsigned", "void", "volatile", "while", NULL
};

int is_keyword(const char* str) {
    for (int i = 0; keywords[i] != NULL; i++) {
        if (strcmp(str, keywords[i]) == 0) {
            return 1;
        }
    }
    return 0;
}

static int peekc() {
    return lex_process->function->peek_char(lex_process);
}

static char nextc() {
    int c = lex_process->function->next_char(lex_process);
    lex_process->pos.col += 1;
    if (c == '\n') {
        lex_process->pos.line += 1;
        lex_process->pos.col = 1;
    }
    return c;
}

static void pushc(char c) {
```

```
    lex_process->function->push_char(lex_process, c);
}

static struct pos lex_file_position() {
    return lex_process->pos;
}

struct token* token_create(struct token* _token) {
    memcpy(&tmp_token, _token, sizeof(struct token));
    tmp_token.pos = lex_file_position();
    return &tmp_token;
}

static struct token* lexer_last_token() {
    return vector_back_or_null(lex_process->token_vec);
}

static struct token* handle_whitespace() {
    struct token* last_token = lexer_last_token();
    if (last_token) {
        last_token->whitespace = true;
    }
    nextc();
    return read_next_token();
}

unsigned long long read_number() {
    struct buffer* buffer = buffer_create();
    char c = peekc();

    if (c == '0') {
        buffer_write(buffer, nextc()); // pega o '0'
        c = peekc();

        if (c == 'x' || c == 'X') {
            buffer_write(buffer, nextc()); // pega o 'x'
            while (isxdigit(peekc())) {
                buffer_write(buffer, nextc());
            }
            buffer_write(buffer, 0x00);
            const char* hex = buffer_ptr(buffer);
            unsigned long long value = strtoull(hex, NULL, 16);
```

```
            printf("Token número hexadecimal: %s (dec: %llu)\n", hex,
value);
            return value;
        } else if (c == 'b' || c == 'B') {
            buffer_write(buffer, nextc()); // pega o 'b'
            while (peekc() == '0' || peekc() == '1') {
                buffer_write(buffer, nextc());
            }
            buffer_write(buffer, 0x00);
            const char* bin = buffer_ptr(buffer);
            unsigned long long value = 0;
            for (int i = 2; bin[i] != '\0'; i++) {
                value = value * 2 + (bin[i] - '0');
            }
                printf("Token número binário: %s (dec: %llu)\n", bin,
value);
            return value;
        } else {
            while (isdigit(peekc())) {
                buffer_write(buffer, nextc());
            }
            buffer_write(buffer, 0x00);
            const char* dec = buffer_ptr(buffer);
            unsigned long long value = strtoull(dec, NULL, 10);
            printf("Token número decimal: %s\n", dec);
            return value;
        }
    } else {
        while (isdigit(peekc())) {
            buffer_write(buffer, nextc());
        }
        buffer_write(buffer, 0x00);
        const char* dec = buffer_ptr(buffer);
        unsigned long long value = strtoull(dec, NULL, 10);
        printf("Token número decimal: %s\n", dec);
        return value;
    }
}

struct token* token_make_number_for_value(unsigned long number) {
         return  token_create(&(struct  token){.type=TOKEN_TYPE_NUMBER,
.llnum=number});
}
```

```c
struct token* token_make_number() {
    return token_make_number_for_value(read_number());
}


const char* read_string() {
    struct buffer* buffer = buffer_create();
    nextc(); // Skip opening quote
    char c = peekc();
    while (c != '"' && c != EOF) {
        buffer_write(buffer, c);
        nextc();
        c = peekc();
    }
    nextc(); // Skip closing quote
    buffer_write(buffer, 0x00);
    printf("Token string: \"%s\"\n", buffer->data);
    return buffer_ptr(buffer);
}

const char* read_comment() {
    struct buffer* buffer = buffer_create();
    char first = nextc();

    if (first == '/') {
        // Comentário de linha
        buffer_write(buffer, '/');
        buffer_write(buffer, '/');
        while (peekc() != '\n' && peekc() != EOF) {
            buffer_write(buffer, nextc());
        }
    } else if (first == '*') {
        // Comentário de bloco
        buffer_write(buffer, '/');
        buffer_write(buffer, '*');
        char c = peekc();
        char next;

        while (1) {
            if (c == EOF) break;

            c = nextc();
```

```
            buffer_write(buffer, c);

            if (c == '*' && peekc() == '/') {
                buffer_write(buffer, nextc()); // pega '/'
                break;
            }

            c = peekc();
        }
    } else {
        // Caso inesperado, como "*/" sem "/*"
        buffer_write(buffer, '/');
        buffer_write(buffer, first);
    }

    buffer_write(buffer, 0x00);
    printf("Token comentário: %s\n", buffer->data);
    return buffer_ptr(buffer);
}


const char* read_preprocessor_directive() {
    struct buffer* buffer = buffer_create();
    char c = peekc();
    while (c != '\n' && c != EOF) {
        buffer_write(buffer, c);
        nextc();
        c = peekc();
    }
    buffer_write(buffer, 0x00);
    printf("Token pré-processador: #%s\n", buffer->data);
    return buffer_ptr(buffer);
}

struct token* read_next_token() {
    struct token* token = NULL;
    char c = peekc();

    switch (c) {
        case EOF:
            return NULL;

        case ' ':
```

```
        case '\t':
            token = handle_whitespace();
            break;

        case '\n':
            nextc();
                                    token    =    token_create(&(struct
token){.type=TOKEN_TYPE_NEWLINE});
            break;

          case ';': case '{': case '}': case '(': case ')': case ',':
case '[': case ']':
            nextc();
                                    token    =    token_create(&(struct
token){.type=TOKEN_TYPE_SYMBOL, .cval=c});
            printf("Token: %c\n", c);
            break;

        case '+': case '-': case '*': case '=':
        case '<': case '>': case '!': case '&': case '|':
            nextc();
                                    token    =    token_create(&(struct
token){.type=TOKEN_TYPE_OPERATOR, .cval=c});
            printf("Token: %c\n", c);
            break;

        case '/':
            nextc();
            if (peekc() == '/' || peekc() == '*') {
                token = token_create(&(struct token){
                    .type = TOKEN_TYPE_COMMENT,
                    .sval = read_comment()
                });
            } else {
                                     token   =   token_create(&(struct
token){.type=TOKEN_TYPE_OPERATOR, .cval='/'});
            }
            break;

        case '"':
                                    token    =    token_create(&(struct
token){.type=TOKEN_TYPE_STRING, .sval=read_string()});
            break;
```

```
            case '#':
            nextc(); // consome o '#'
            token = token_create(&(struct token){
                .type = TOKEN_TYPE_SYMBOL,
                .cval = '#'
            });
            printf("Token: %c\n", '#');
             vector_push(lex_process->token_vec, token); // adiciona '#'
separado

            // Agora lemos a palavra que vem depois do #
              while (peekc() == ' ' || peekc() == '\t') nextc(); // pula
espaços
            {
                struct buffer* buf = buffer_create();
                while (isalnum(peekc()) || peekc() == '_') {
                    buffer_write(buf, nextc());
                }
                buffer_write(buf, 0x00);
                const char* id = buffer_ptr(buf);
                                        token   =   token_create(&(struct
token){.type=TOKEN_TYPE_IDENTIFIER, .sval=id});
                printf("Token: %s\n", id);
                 vector_push(lex_process->token_vec, token); // adiciona
o include por exemplo
            }

            // Agora o argumento, como <stdio.h> ou "meu.h"
              while (peekc() == ' ' || peekc() == '\t') nextc(); // pula
espaços
            {
                struct buffer* buf = buffer_create();
                char c = peekc();
                if (c == '<') {
                    buffer_write(buf, nextc()); // escreve '<'
                    while ((c = peekc()) != '>' && c != EOF) {
                        buffer_write(buf, nextc());
                    }
                      if (peekc() == '>') buffer_write(buf, nextc()); //
fecha com '>'
                } else if (c == '"') {
                    buffer_write(buf, nextc()); // escreve '"'
```

```
                    while ((c = peekc()) != '"' && c != EOF) {
                        buffer_write(buf, nextc());
                    }
                     if (peekc() == '"') buffer_write(buf, nextc()); //
fecha com '"'
                } else {
                    // apenas lê como identificador
                    while (!isspace(peekc()) && peekc() != EOF) {
                        buffer_write(buf, nextc());
                    }
                }
                buffer_write(buf, 0x00);
                const char* arg = buffer_ptr(buf);
                                        token   =   token_create(&(struct
token){.type=TOKEN_TYPE_IDENTIFIER, .sval=arg});
                printf("Token: %s\n", arg);
            }
            break;


        default:
            if (isdigit(c)) {
                token = token_make_number();
            } else if (isalpha(c) || c == '_') {
                struct buffer* buf = buffer_create();
                while (isalnum(peekc()) || peekc() == '_') {
                    buffer_write(buf, nextc());
                }
                buffer_write(buf, 0x00);
                const char* id = buffer_ptr(buf);
                if (is_keyword(id)) {
                                        token   =   token_create(&(struct
token){.type=TOKEN_TYPE_KEYWORD, .sval=id});
                    printf("Token palavra reservada: %s\n", id);
                } else {
                                        token   =   token_create(&(struct
token){.type=TOKEN_TYPE_IDENTIFIER, .sval=id});
                    printf("Token identificador: %s\n", id);
                }
            } else {
                printf("Caractere não reconhecido: '%c'\n", c);
                nextc();
            }
```

```
            break;
    }

    return token;
}

int lex(struct lex_process* process) {
    process->current_expression_count = 0;
    process->parentheses_buffer = NULL;
    lex_process = process;
    process->pos.filename = process->compiler->cfile.abs_path;

    struct token* token = read_next_token();
    while (token) {
        vector_push(process->token_vec, token);
        token = read_next_token();
    }

    return LEXICAL_ANALYSIS_ALL_OK;
}
/* END - LAB 2 ------------------------------------*/
```

compiler.h

```
#ifndef COMPILER_H
#define COMPILER_H

#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <string.h> // Se necessário para memcpy

// Declarações antecipadas para evitar erros de tipo incompleto
struct compile_process;
struct token;
struct node;
struct vector;
struct buffer;

/* ----------------- LAB 2 ----------------- */
```

```c
struct pos {
    int line;
    int col;
    const char* filename;
};

#define TOKEN_TYPE_PREPROCESSOR 10
#define S_EQ(a, b) (strcmp((a), (b)) == 0)


#define NUMERIC_CASE \
    case '0': \
    case '1': \
    case '2': \
    case '3': \
    case '4': \
    case '5': \
    case '6': \
    case '7': \
    case '8': \
    case '9'

enum {
    LEXICAL_ANALYSIS_ALL_OK,
    LEXICAL_ANALYSIS_IMPUT_ERROR
};

enum {
    TOKEN_TYPE_KEYWORD,
    TOKEN_TYPE_IDENTIFIER,
    TOKEN_TYPE_OPERATOR,
    TOKEN_TYPE_SYMBOL,
    TOKEN_TYPE_NUMBER,
    TOKEN_TYPE_STRING,
    TOKEN_TYPE_COMMENT,
    TOKEN_TYPE_NEWLINE
};

// Flags para nós sintáticos (LAB3)
enum {
    NODE_FLAG_INSIDE_EXPRESSION = 0b00000001
};
```

```c
/* --- FUNÇÕES parser.c --- */
int parse(struct compile_process* process);
int parse_next();

/* --- FUNÇÕES token.c --- */
bool token_is_keyword(struct token* token, const char* value);
bool token_is_symbol(struct token* token, const char value);
bool discart_token(struct token* token);

/* --- FUNÇÕES node.c --- */
void node_set_vector(struct vector* vec, struct vector* root_vec);
void node_push(struct node* node);
struct node* node_peek_or_null();
struct node* node_peek();
struct node* node_pop();
struct node* node_peek_expressionable_or_null();
bool node_is_expressionable(struct node* node);
void  make_exp_node(struct  node*  node_left,  struct  node*  node_right,
const char* op);
struct node* node_create(struct node* _node);

// Estrutura de token
struct token {
    int type;
    int flags;
    struct pos pos;

    union {
        char cval;
        const char* sval;
        unsigned int inum;
        unsigned long lnum;
        unsigned long long llnum;
        void* any;
    };

    bool whitespace;
    const char* between_brackets;
};

struct lex_process;
```

```
typedef int (*LEX_PROCESS_NEXT_CHAR)(struct lex_process* process);
typedef int (*LEX_PROCESS_PEEK_CHAR)(struct lex_process* process);
typedef void (*LEX_PROCESS_PUSH_CHAR)(struct lex_process* process, char
c);
typedef char (*LEX_PROCESS_PEEK_AHEAD)(struct lex_process* process, int
offset);

struct lex_process_functions {
    LEX_PROCESS_NEXT_CHAR next_char;
    LEX_PROCESS_PEEK_CHAR peek_char;
    LEX_PROCESS_PUSH_CHAR push_char;
    LEX_PROCESS_PEEK_AHEAD peek_ahead;
};

struct lex_process {
    struct pos pos;
    struct vector* token_vec;
    struct compile_process* compiler;
    int current_expression_count;
    struct buffer* parentheses_buffer;
    struct lex_process_functions* function;
    void* private;
    const char* input;
    int input_size;
};

// cprocess.c
int compile_process_next_char(struct lex_process* lex_process);
int compile_process_peek_char(struct lex_process* lex_process);
void  compile_process_push_char(struct  lex_process*  lex_process,  char
c);

// lex_process.c
struct    lex_process*    lex_process_create(struct    compile_process*
compiler, struct lex_process_functions* functions, void* private);
void lex_process_free(struct lex_process* process);
void* lex_process_private(struct lex_process* process);
struct vector* lex_process_tokens(struct lex_process* process);

// lexer.c
int lex(struct lex_process* process);

// compiler.c
```

```
void compiler_error(struct compile_process* compiler, const char* msg, ...);
void compiler_warning(struct compile_process* compiler, const char* msg, ...);

/* ----------------- LAB 3 ----------------- */

enum {
    COMPILER_FILE_COMPILED_OK,
    COMPILER_FAILED_WITH_ERRORS
};

struct compile_process {
    int flags;
    struct pos pos;

    struct compile_process_input_file {
        FILE* fp;
        const char* abs_path;
    } cfile;

    struct vector* token_vec;
    struct vector* node_vec;
    struct vector* node_tree_vec;

    FILE* ofile;
};

enum {
    NODE_TYPE_EXPRESSION,
    NODE_TYPE_EXPRESSION_PARENTHESES,
    NODE_TYPE_NUMBER,
    NODE_TYPE_IDENTIFIER,
    NODE_TYPE_STRING,
    NODE_TYPE_VARIABLE,
    NODE_TYPE_VARIABLE_LIST,
    NODE_TYPE_FUNCTION,
    NODE_TYPE_BODY,
    NODE_TYPE_STATEMENT_RETURN,
    NODE_TYPE_STATEMENT_IF,
    NODE_TYPE_STATEMENT_ELSE,
    NODE_TYPE_STATEMENT_WHILE,
    NODE_TYPE_STATEMENT_DO_WHILE,
```

```
    NODE_TYPE_STATEMENT_FOR,
    NODE_TYPE_STATEMENT_BREAK,
    NODE_TYPE_STATEMENT_CONTINUE,
    NODE_TYPE_STATEMENT_SWITCH,
    NODE_TYPE_STATEMENT_CASE,
    NODE_TYPE_STATEMENT_DEFAULT,
    NODE_TYPE_STATEMENT_GOTO,
    NODE_TYPE_UNARY,
    NODE_TYPE_TENARY,
    NODE_TYPE_LABEL,
    NODE_TYPE_STRUCT,
    NODE_TYPE_UNION,
    NODE_TYPE_BRACKET,
    NODE_TYPE_CAST,
    NODE_TYPE_BLANK,
};

enum {
    PARSE_ALL_OK,
    PARSE_GENERAL_ERROR
};

// Estrutura de um nó sintático
struct node {
    int type;
    int flags;
    struct pos pos;

    struct node_binded {
        struct node* owner;
        struct node* funtion;
    } binded;

    union {
        char cval;
        const char* sval;
        unsigned int inum;
        unsigned long lnum;
        unsigned long long llnum;
        void* any;
    };

    union {
```

```
        struct exp {
            struct node* left;
            struct node* right;
            const char* op;
        } exp;
    };
};

// compiler.c
int compile_file(const char* filename, const char* out_filename, int
flags);

#endif // COMPILER_H
```

token.c

```
#include "compiler.h"
bool token_is_keyword(struct token* token, const char* value) {
return token->type == TOKEN_TYPE_KEYWORD && S_EQ(token->sval, value);
}
bool token_is_symbol(struct token* token, char value) {
return (token->type == TOKEN_TYPE_SYMBOL) && (token->cval == value);
}
bool discart_token(struct token* token) {
return (token->type == TOKEN_TYPE_NEWLINE) ||
(token->type == TOKEN_TYPE_COMMENT) ||
token_is_symbol(token, '\\');
}
```

node.c

```
#include "compiler.h"
#include "helpers/vector.h"
#include <assert.h>

struct vector* node_vector = NULL;
struct vector* node_vector_root = NULL;
```

```c
// Inicializa os ponteiros globais desse segmento de código.
void node_set_vector(struct vector* vec, struct vector* root_vec) {
    node_vector = vec;
    node_vector_root = root_vec;
}

// Adiciona o node ao final da lista.
void node_push(struct node* node) {
    vector_push(node_vector, &node);
}

// Pega o node da parte final do vetor. Se não tiver nada, retorna NULL.
struct node* node_peek_or_null() {
    return vector_back_ptr_or_null(node_vector);
}

struct node* node_peek() {
    return *(struct node**) vector_back(node_vector);
}

struct node* node_pop() {
    struct node* last_node = vector_back_ptr(node_vector);
     struct node* last_node_root = vector_empty(node_vector) ? NULL : vector_back_ptr_or_null(node_vector_root);

    vector_pop(node_vector);

    if (last_node == last_node_root)
        vector_pop(node_vector_root);

    return last_node;
}

// Tipos de nós que podem ser colocados dentro de uma expressão.
bool node_is_expressionable(struct node* node) {
    return node->type == NODE_TYPE_EXPRESSION ||
           node->type == NODE_TYPE_EXPRESSION_PARENTHESES ||
           node->type == NODE_TYPE_UNARY ||
           node->type == NODE_TYPE_IDENTIFIER ||
           node->type == NODE_TYPE_NUMBER ||
           node->type == NODE_TYPE_STRING;
}
```

```
// Verifica se o node informado pode ser colocado dentro de uma expressão.
struct node* node_peek_expressionable_or_null() {
    struct node* last_node = node_peek_or_null();
    return node_is_expressionable(last_node) ? last_node : NULL;
}

// Cria um node de expressão.
void make_exp_node(struct node* node_left, struct node* node_right, const char* op) {
    assert(node_left);
    assert(node_right);
    node_create(&(struct node){
        .type = NODE_TYPE_EXPRESSION,
        .exp.left = node_left,
        .exp.right = node_right,
        .exp.op = op
    });
}

// Função para criar um node.
struct node* node_create(struct node* _node) {
    struct node* node = malloc(sizeof(struct node));
    memcpy(node, _node, sizeof(struct node));
    node_push(node);
    return node;
}
```

parser.c

```
#include "compiler.h"
#include "helpers/vector.h"

static struct compile_process* current_process;
static struct token* parser_last_token;

// Estrutura para passar comandos através de funções recursivas.
struct history {
```

```
    int flags;
};

void parse_expressionable(struct history* history);
int parse_expressionable_single(struct history* history);

// Inicia um novo histórico com flags.
struct history* history_begin(int flags) {
    struct history* history = calloc(1, sizeof(struct history));
    history->flags = flags;
    return history;
}

// Cria um novo histórico baseado em outro.
struct history* history_down(struct history* history, int flags) {
    struct history* new_history = calloc(1, sizeof(struct history));
    memcpy(new_history, history, sizeof(struct history));
    new_history->flags = flags;
    return new_history;
}

// Ignora tokens do tipo comentário ou nova linha.
static void parser_ignore_nl_or_comment(struct token* token) {
    while (token && discart_token(token)) {
        // Pula o token que deve ser descartado no vetor.
        vector_peek(current_process->token_vec);
        // Pega o próximo token para ver se ele também será descartado.
        token = vector_peek_no_increment(current_process->token_vec);
    }
}

// Recupera o próximo token, pulando comentários e novas linhas.
static struct token* token_next() {
                            struct       token*       next_token       =
vector_peek_no_increment(current_process->token_vec);
    parser_ignore_nl_or_comment(next_token);
     current_process->pos = next_token->pos; // Atualiza a posição do
arquivo de compilação.
    parser_last_token = next_token;
    return vector_peek(current_process->token_vec);
}

// Apenas visualiza o próximo token sem consumir.
```

```c
static struct token* token_peek_next() {
                            struct       token*       next_token       =
vector_peek_no_increment(current_process->token_vec);
    parser_ignore_nl_or_comment(next_token);
    return vector_peek_no_increment(current_process->token_vec);
}

// Converte um token simples diretamente para nó.
void parse_single_token_to_node() {
    struct token* token = token_next();
    struct node* node = NULL;

    switch (token->type) {
        case TOKEN_TYPE_NUMBER:
                            node  =  node_create(&(struct  node){  .type  =
NODE_TYPE_NUMBER, .llnum = token->llnum });
            break;
        case TOKEN_TYPE_IDENTIFIER:
                            node  =  node_create(&(struct  node){  .type  =
NODE_TYPE_IDENTIFIER, .sval = token->sval });
            break;
        case TOKEN_TYPE_STRING:
                            node  =  node_create(&(struct  node){  .type  =
NODE_TYPE_STRING, .sval = token->sval });
            break;
        default:
                compiler_error(current_process, "Esse token não pode ser
convertido para node!\n");
            break;
    }
}

// Inicia análise de expressão com operador.
void  parse_expressionable_for_op(struct  history*  history,  const  char*
op) {
    parse_expressionable(history);
}

// Faz o parsing de uma expressão binária normal.
void parse_exp_normal(struct history* history) {
    struct token* op_token = token_peek_next();
    const char* op = op_token->sval;
```

```
    struct node* node_left = node_peek_expressionable_or_null();
    if (!node_left) return;

    // Consome o token e o node
    token_next();
    node_pop();

    node_left->flags |= NODE_FLAG_INSIDE_EXPRESSION;

     parse_expressionable_for_op(history_down(history, history->flags),
op);

    struct node* node_right = node_pop();
    node_right->flags |= NODE_FLAG_INSIDE_EXPRESSION;

    make_exp_node(node_left, node_right, op);

    struct node* exp_node = node_pop();
       // TODO: Inserir aqui código para reordenar a expressão se
necessário.
    node_push(exp_node);
}

// Inicia o parsing de uma expressão com operador.
int parse_exp(struct history* history) {
    parse_exp_normal(history);
    return 0;
}

// Analisa um único token que pode fazer parte de uma expressão.
int parse_expressionable_single(struct history* history) {
    struct token* token = token_peek_next();
    if (!token) return -1;

    history->flags |= NODE_FLAG_INSIDE_EXPRESSION;
    int res = -1;

    switch (token->type) {
        case TOKEN_TYPE_NUMBER:
            parse_single_token_to_node();
            res = 0;
            break;
```

```
        case TOKEN_TYPE_OPERATOR:
            parse_exp(history);
            res = 0;
            break;
        default:
            break;
    }

    return res;
}

// Analisa uma sequência de tokens expressáveis.
void parse_expressionable(struct history* history) {
    while (parse_expressionable_single(history) == 0) {}
}

// Processa o próximo elemento do código-fonte.
int parse_next() {
    struct token* token = token_peek_next();
    if (!token) return -1;

    switch (token->type) {
        case TOKEN_TYPE_NUMBER:
        case TOKEN_TYPE_IDENTIFIER:
        case TOKEN_TYPE_STRING:
            parse_expressionable(history_begin(0));
            break;
        default:
            break;
    }

    return 0;
}

// Função principal de parsing.
int parse(struct compile_process* process) {
    current_process = process;
    parser_last_token = NULL;
    struct node* node = NULL;

    node_set_vector(process->node_vec, process->node_tree_vec);
    vector_set_peek_pointer(process->token_vec, 0);
```

```
    while (parse_next() == 0) {
        node = node_peek();
        vector_push(process->node_tree_vec, &node);
    }

    return PARSE_ALL_OK;
}
```

makefile

```
OBJECTS=    ./build/main.o    ./build/compiler.o    ./build/cprocess.o
./build/helpers/buffer.o ./build/helpers/vector.o \
               ./build/teste1.o  ./build/lex_process.o  ./build/lexer.o
./build/parser.o ./build/node.o ./build/token.o


INCLUDES= -I./

all: ${OBJECTS}
    gcc ${OBJECTS} -g -o ./main

./build/main.o: ./main.c
    gcc ./main.c ${INCLUDES} -o ./build/main.o -g -c

./build/compiler.o: ./compiler.c
    gcc ./compiler.c ${INCLUDES} -o ./build/compiler.o -g -c

./build/cprocess.o: ./cprocess.c
    gcc ./cprocess.c ${INCLUDES} -o ./build/cprocess.o -g -c

./build/helpers/buffer.o: ./helpers/buffer.c
     gcc ./helpers/buffer.c ${INCLUDES} -o ./build/helpers/buffer.o -g
-c

./build/helpers/vector.o: ./helpers/vector.c
     gcc ./helpers/vector.c ${INCLUDES} -o ./build/helpers/vector.o -g
-c

./build/teste1.o: ./teste1.c
    gcc ./teste1.c ${INCLUDES} -o ./build/teste1.o -g -c
```

```
./build/lex_process.o: ./lex_process.c
    gcc ./lex_process.c ${INCLUDES} -o ./build/lex_process.o -g -c

./build/lexer.o: ./lexer.c
    gcc ./lexer.c ${INCLUDES} -o ./build/lexer.o -g -c

./build/parser.o: ./parser.c
    gcc ./parser.c ${INCLUDES} -o ./build/parser.o -g -c

./build/node.o: ./node.c
    gcc ./node.c ${INCLUDES} -o ./build/node.o -g -c

./build/token.o: ./token.c
    gcc ./token.c ${INCLUDES} -o ./build/token.o -g -c

clean:
    rm -f ./main
    rm -f ${OBJECTS}
```

main.c

```
#include <stdio.h>
#include "helpers/vector.h"
#include "helpers/buffer.h"
#include "compiler.h"
#include "cprocess.h"

int main() {
    printf("Compiladores - TURMA A/B - GRUPO X\n\n");

    int res = compile_file("./teste.c", "./outtest", 0);
    if (res == COMPILER_FILE_COMPILED_OK) {
        printf("Todos os arquivos foram compilados com sucesso!\n");
    } else if (res == COMPILER_FAILED_WITH_ERRORS) {
        printf("Erro de compilacao!\n");
    } else {
        printf("Erro desconhecido!\n");
    }
```

```
    return 0;
}
```

teste.c (primeiro teste)

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int a = 0x1A3F;       // hexadecimal
    int b = 0b10101010;   // binário
    int c = 123;          // decimal normal
     float d = 0x10;        // float com valor hexadecimal (vai virar
decimal 16)
    float e = 0b11;       // binário como float
    return 0;
}
```

root@LAPTOP-P06AHGNA:~/compiler# make clean
rm -f ./main
rm -f ./build/main.o ./build/compiler.o ./build/cprocess.o ./build/helpers/buffer.o ./build/helpers/vector.o ./build/teste1.o ./build/lex_process.o ./build/lexer.o ./build/parser.o ./build/node.o ./build/token.o
root@LAPTOP-P06AHGNA:~/compiler# make all
gcc ./main.c -I./ -o ./build/main.o -g -c
gcc ./compiler.c -I./ -o ./build/compiler.o -g -c
gcc ./cprocess.c -I./ -o ./build/cprocess.o -g -c
gcc ./helpers/buffer.c -I./ -o ./build/helpers/buffer.o -g -c
gcc ./helpers/vector.c -I./ -o ./build/helpers/vector.o -g -c
gcc ./teste1.c -I./ -o ./build/teste1.o -g -c
gcc ./lex_process.c -I./ -o ./build/lex_process.o -g -c
gcc ./lexer.c -I./ -o ./build/lexer.o -g -c
gcc ./parser.c -I./ -o ./build/parser.o -g -c
gcc ./node.c -I./ -o ./build/node.o -g -c
gcc ./token.c -I./ -o ./build/token.o -g -c

```
gcc ./build/main.o ./build/compiler.o ./build/cprocess.o ./build/helpers/buffer.o ./build/helpers/vector.o ./build/teste1.o ./build/lex_process.o ./build/lexer.o ./build/parser.o ./build/node.o ./build/token.o -g -o ./main
root@LAPTOP-P06AHGNA:~/compiler# ./main
Compiladores – TURMA A/B – GRUPO X

Token: #
Token: include
Token: <stdio.h>
Token: #
Token: include
Token: <stdlib.h>
Token palavra reservada: int
Token identificador: main
Token: (
Token: )
Token: {
Token palavra reservada: int
Token identificador: a
Token: =
Token número hexadecimal: 0x1A3F (dec: 6719)
Token: ;
Token comentário: // hexadecimal
Token palavra reservada: int
Token identificador: b
Token: =
Token número binário: 0b10101010 (dec: 170)
Token: ;
Token comentário: // binário
Token palavra reservada: int
Token identificador: c
Token: =
Token número decimal: 123
Token: ;
Token comentário: // decimal normal
Token palavra reservada: float
Token identificador: d
```

Token: =
Token número hexadecimal: 0x10 (dec: 16)
Token: ;
Token comentário: // float com valor hexadecimal (vai virar decimal 16)
Token palavra reservada: float
Token identificador: e
Token: =
Token número binário: 0b11 (dec: 3)
Token: ;
Token comentário: // binário como float
Token palavra reservada: return
Token número decimal: 0
Token: ;
Token: }
Todos os arquivos foram compilados com sucesso!

SEGUNDO TESTE:

“Fazer o teste com arquivo de entrada (test.c): 1234 + 6789”

main.c

```c
#include <stdio.h>
#include "helpers/vector.h"
#include "helpers/buffer.h"
#include "compiler.h"
#include "cprocess.h"

int main() {
    printf("Compiladores - TURMA A/B - GRUPO X\n\n");

    int res = compile_file("./test.c", "./outtest", 0);
    if (res == COMPILER_FILE_COMPILED_OK) {
        printf("Todos os arquivos foram compilados com sucesso!\n");
    } else if (res == COMPILER_FAILED_WITH_ERRORS) {
        printf("Erro de compilacao!\n");
    } else {
        printf("Erro desconhecido!\n");
    }
```

```c
    return 0;
}
```

```
root@LAPTOP-P06AHGNA:~/compiler# make clean
rm -f ./main
rm -f ./build/main.o ./build/compiler.o ./build/cprocess.o ./build/helpers/buffer.o ./build/helpers/vector.o ./build/teste1.o ./build/lex_process.o ./build/lexer.o ./build/
parser.o ./build/node.o ./build/token.o
root@LAPTOP-P06AHGNA:~/compiler# make all
gcc ./main.c -I./ -o ./build/main.o -g -c
gcc ./compiler.c -I./ -o ./build/compiler.o -g -c
gcc ./cprocess.c -I./ -o ./build/cprocess.o -g -c
gcc ./helpers/buffer.c -I./ -o ./build/helpers/buffer.o -g -c
gcc ./helpers/vector.c -I./ -o ./build/helpers/vector.o -g -c
gcc ./teste1.c -I./ -o ./build/teste1.o -g -c
gcc ./lex_process.c -I./ -o ./build/lex_process.o -g -c
gcc ./lexer.c -I./ -o ./build/lexer.o -g -c
gcc ./parser.c -I./ -o ./build/parser.o -g -c
gcc ./node.c -I./ -o ./build/node.o -g -c
gcc ./token.c -I./ -o ./build/token.o -g -c
gcc ./build/main.o ./build/compiler.o ./build/cprocess.o ./build/helpers/buffer.o ./build/helpers/vector.o ./build/teste1.o ./build/lex_process.o ./build/lexer.o ./build/pa
rser.o ./build/node.o ./build/token.o -g -o ./main
root@LAPTOP-P06AHGNA:~/compiler# ./main
Compiladores - TURMA A/B - GRUPO X

Token número decimal: 1234
Token: +
Token número decimal: 6789
Todos os arquivos foram compilados com sucesso!
root@LAPTOP-P06AHGNA:~/compiler#
```